Министерство науки и высшего образования Российской Федерации
ФГАОУ ВО «Уральский федеральный университет
имени первого Президента России Б.Н. Ельцина»
Институт новых материалов и технологий
Кафедра «Теплофизика и информатика в металлургии»

Сборка и запуск контейнера в Docker

ОТЧЕТ
по практической работе № 1
по дисциплине «Основы методологии Development Operation»

Направление 09.03.02 «Информационные системы и технологии» (уровень бакалавриата)
Образовательная программа
09.03.02/33.02 «Информационные системы и технологии» (СУОС)

Студент
группы НМТ-433901 Малышева О.М.

Преподаватель:
профессор, д.т.н. В.В. Лавров

Екатеринбург
2026

СОДЕРЖАНИЕ

Практическая работа «Сборка и запуск контейнера в Docker» 3
1.1 Цель работы 3
1.2 Ход проведения работы 3
1.2.1 Разработка тестового приложения Visual Studio .NET Core 3
1.2.2 Создание контейнера с использованием Docker 3
1.2.3 Размещение приложения в системе контроля версий GitHub 4
1.2.4 Сборка образа с использованием команды docker build 4
1.2.5 Создание контейнера с использованием Docker-образа и запуск его 5
1.2.6 Демонстрация возможности публикации и остановки контейнера 5
1.3 Выводы 5
Приложение А Листинг программного кода приложения «Калькулятор» (контроллер) 7
Приложение Б Листинг программного кода приложения «Калькулятор» (appsettings.json) 8
Приложение В Листинг программного кода приложения «Калькулятор» (appsettings.Development.json) 9

Практическая работа «Сборка и запуск контейнера в Docker»

1.1 Цель работы

Цель практической работы — разработать простое веб-приложение «Калькулятор», подготовить Dockerfile, разместить исходный код в GitHub, собрать Docker-образ и запустить приложение в контейнере на удалённом сервере.

В ходе выполнения работы были освоены основные этапы контейнеризации веб-приложения: подготовка проекта, сборка образа, публикация изменений в репозитории и запуск контейнера с пробросом порта для доступа через браузер.

1.2 Ход проведения работы

Работа выполнялась последовательно: сначала было подготовлено веб-приложение, затем настроена сборка Docker-образа, после чего код синхронизирован с GitHub и контейнер запущен на сервере.

1.2.1 Разработка тестового приложения Visual Studio .NET Core

В Visual Studio создано веб-приложение «Calculator» на платформе ASP.NET Core. Интерфейс приложения содержит поля для ввода первого и второго числа, список выбора арифметической операции и кнопку «Рассчитать». Страница оформлена с помощью HTML, CSS и Bootstrap.

После запуска в локальной среде была проверена загрузка главной страницы и отображение формы калькулятора. Это позволило убедиться, что приложение готово к дальнейшей упаковке в контейнер.

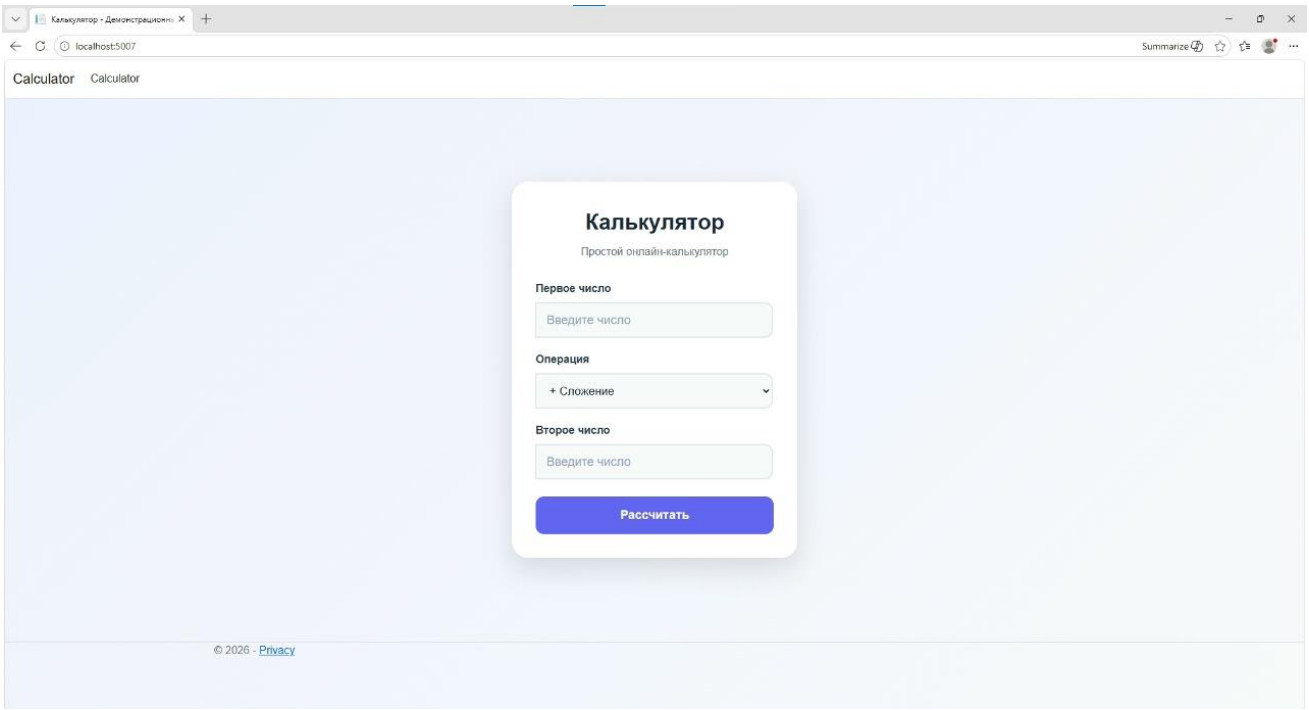


Рисунок 1 — Главная страница веб-приложения «Калькулятор» в локальной среде

1.2.2 Создание контейнера с использованием Docker

Для проекта подготовлен Dockerfile, описывающий сборку приложения и запуск опубликованной версии в контейнере. Dockerfile позволяет повторять сборку в одинаковой последовательности и не зависит от ручного запуска проекта из Visual Studio.

При подготовке контейнера важно указать корректный рабочий каталог, скопировать необходимые файлы проекта, выполнить публикацию приложения и определить команду запуска. В результате Docker может создать образ, содержащий всё необходимое для работы веб-приложения.

1.2.3 Размещение приложения в системе контроля версий GitHub

Исходный код проекта размещён в удалённом репозитории GitHub. После изменения файла представления _Layout.cshtml изменения были зафиксированы коммитом и отправлены в удалённый репозиторий.

На сервере выполнена команда git pull, которая получила последние изменения из ветки master. Таким образом, перед повторной сборкой контейнера серверный проект был синхронизирован с репозиторием.

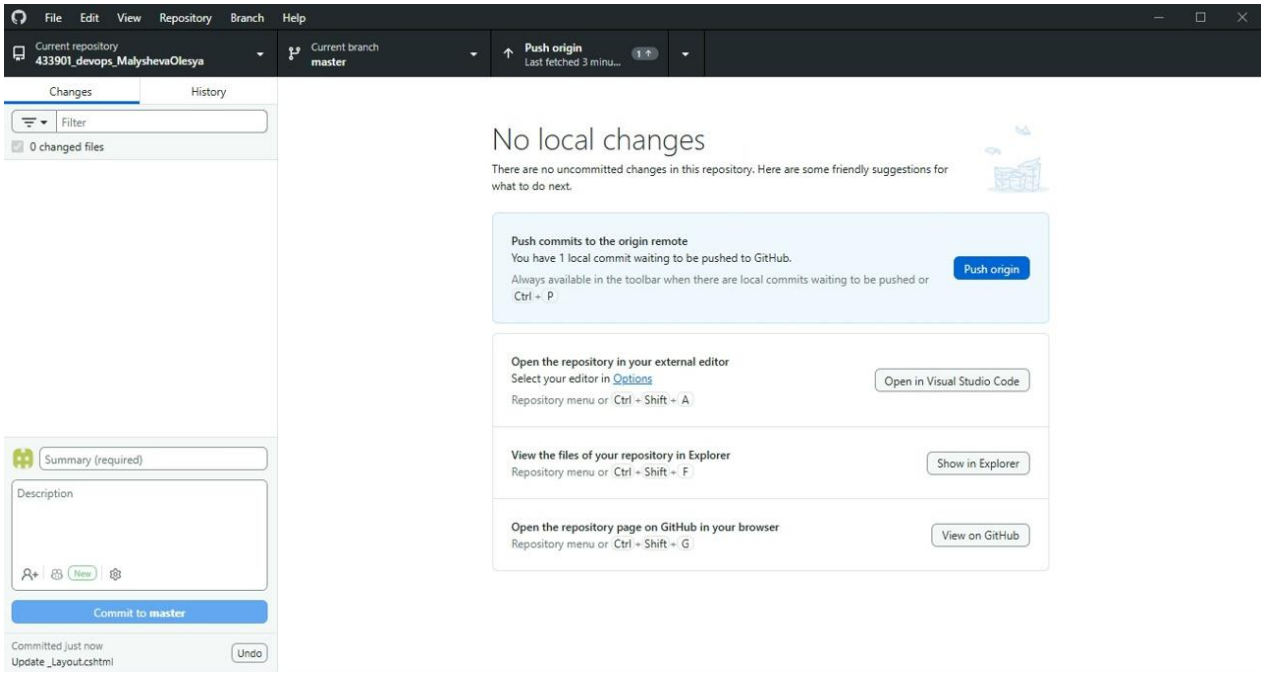


Рисунок 2 — Фиксация изменений и подготовка отправки коммита в GitHub

1.2.4 Сборка образа с использованием команды docker build

В каталоге проекта выполнена сборка образа командой:

```
sudo docker build -t calculator:latest
```

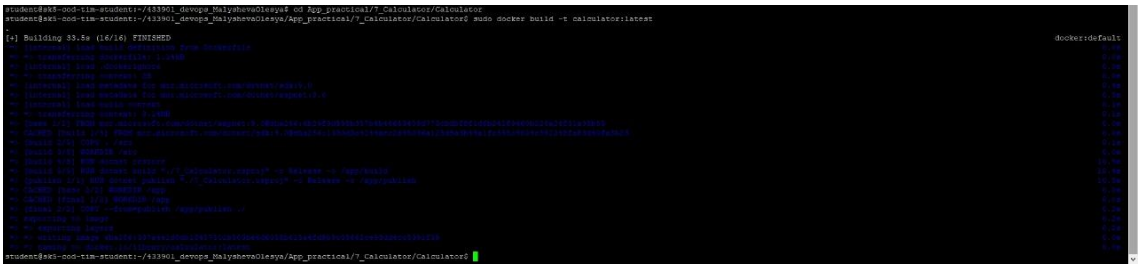


Рисунок 3 — Результат сборки Docker-образа calculator:latest

По результатам выполнения команда завершилась успешно: Docker выполнил этапы сборки и сформировал образ calculator:latest. Сообщение об успешном завершении сборки подтверждает, что Dockerfile и контекст сборки были обработаны без критических ошибок.

1.2.5 Создание контейнера с использованием Docker-образа и запуск его

На удалённом сервере выполнен запуск контейнера из созданного образа с публикацией порта приложения. Для проверки использовались команды остановки и удаления предыдущего контейнера, запуска нового экземпляра и просмотра списка работающих контейнеров.

Пример команды запуска, использованной для публикации приложения на порту 5007:

```
sudo docker run -d -p 5007:5055 --name calculator_olesya_5007 calculator:latest
```



Рисунок 4 — Запуск контейнера и проверка опубликованного порта командой docker ps

После запуска проверено состояние контейнера командой docker ps. В выводе отображается работающий контейнер и публикация внешнего порта 5007 на порт 5055 внутри контейнера согласно использованной команде запуска.

1.2.6 Демонстрация возможности публикации и остановки контейнера

После запуска приложения выполнена проверка через браузер по адресу http://93.88.178.186:5007. Страница калькулятора открывается с удалённого сервера, что подтверждает доступность приложения извне и корректную публикацию порта.

Для управления контейнером использовались команды docker stop и docker rm. При необходимости контейнер можно остановить, удалить и создать заново из собранного образа. Это демонстрирует базовый цикл управления контейнеризированным приложением.

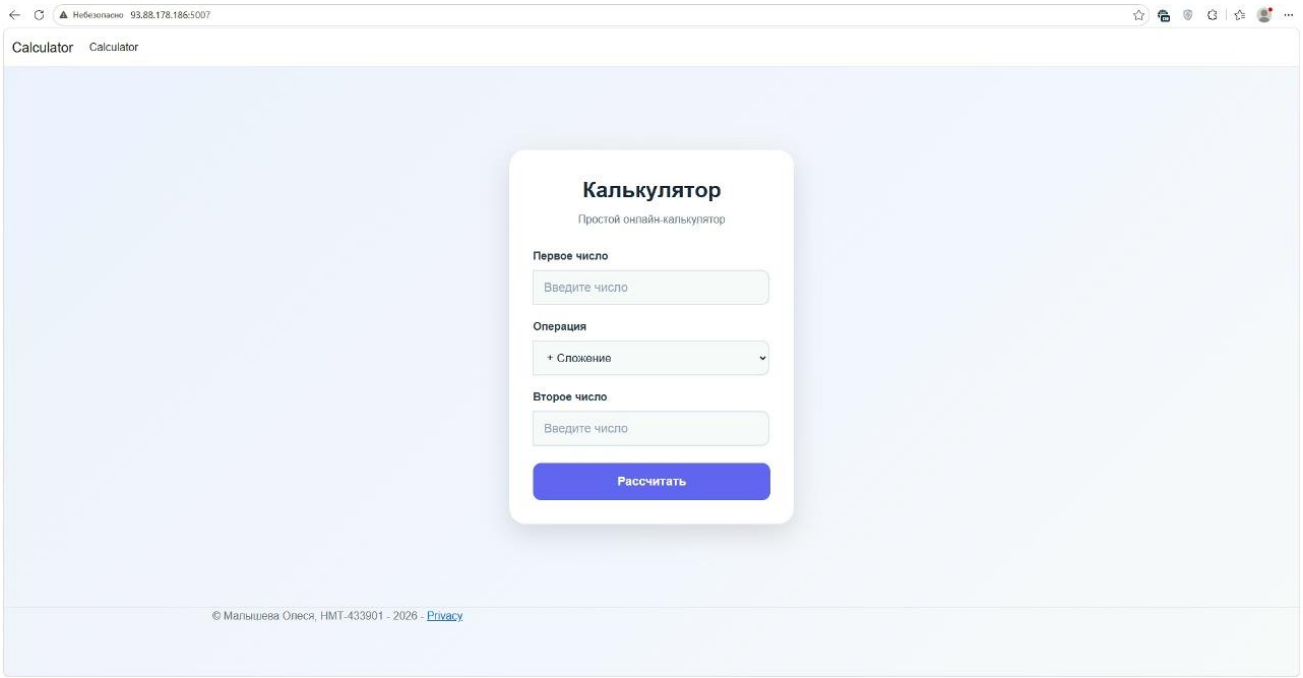


Рисунок 5 — Проверка доступности веб-приложения «Калькулятор» на удалённом сервере

1.3 Выводы

В ходе практической работы было разработано веб-приложение «Калькулятор» и проверена его работа в локальной среде. Для проекта подготовлен Dockerfile, создан Docker-образ командой docker build, а исходный код размещён в GitHub.

Затем проект был обновлён на удалённом сервере, образ собран и контейнер запущен с публикацией порта 5007. Проверка приложения в браузере подтвердила успешный запуск.

Поставленная цель достигнута: приложение доступно через веб-браузер и может управляться средствами Docker.

Приложение А Листинг программного кода приложения «Калькулятор» (контроллер)

```
using Calculator.Data;
using Calculator.Models;
using Microsoft.AspNetCore.Components.Forms;
using Microsoft.AspNetCore.Mvc;

namespace Calculator.Controllers
{
    public enum Operation { Add, Subtract, Multiply, Divide }

    public class CalculatorController : Controller
    {
        private CalculatorContext _context;

        public CalculatorController(CalculatorContext context)
        {
            _context = context;
        }

        [HttpGet]
        public IActionResult Index()
        {
            return View();
        }

        [HttpPost]
        [ValidateAntiForgeryToken]
        public IActionResult Calculate(double num1, double num2, Operation operation)
        {
            double result = 0;
            switch (operation)
            {
                case Operation.Add:
                    result = num1 + num2;
                    break;
                case Operation.Subtract:
                    result = num1 - num2;
                    break;
                case Operation.Multiply:
                    result = num1 * num2;
                    break;
                case Operation.Divide:
                    result = num1 / num2;
                    break;
            }
            ViewBag.Result = result;

            DataInputVariant dataInputVariant = new DataInputVariant();
            dataInputVariant.Operand_1 = num1.ToString();
            dataInputVariant.Operand_2 = num2.ToString();
            dataInputVariant.Type_operation = operation.ToString();
            dataInputVariant.Result = result.ToString();

            //_context.DataInputVariants.Add(dataInputVariant);
            //_context.SaveChanges();

            return View("Index");
        }
    }
}
```

Приложение Б Листинг программного кода приложения «Калькулятор» (appsettings.json)

```
{
  "ConnectionStrings": {
    "DefaultConnection": "Server=mariadb; Database=myWebCalculatorDb; Uid=root; Pwd=password; Character
Set=utf8; ConvertZeroDatetime=True;"
  },
  "Kestrel": {
    "Endpoints": {
      "Http": {
        "Url": "http://0.0.0.0:5007" // Заменить последние 2 цифры порта на порядковый номер из студенче-
ского журнала. Например, порт 5012 соответствует номеру 12
      }
    }
  },
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft": "Warning",
      "Microsoft.Hosting.Lifetime": "Information"
    }
  },
  "AllowedHosts": "*"
}
```

Приложение В Листинг программного кода приложения «Калькулятор» (appsettings.Development.json)

```
{
  "ConnectionStrings": {
    "DefaultConnection": "Server=93.88.178.186; Port=5023; Database=myWebCalculatorDb; Uid=root;
Pwd=password; Character Set=utf8; ConvertZeroDatetime=True;"
  },
  "DetailedErrors": true,
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft": "Warning",
      "Microsoft.Hosting.Lifetime": "Information"
    }
  }
}
```

Приложение Г Листинг программного кода приложения «Калькулятор» (Program.cs)

```
using Microsoft.EntityFrameworkCore;
using Calculator.Data;

var builder = WebApplication.CreateBuilder(args);

// Добавление сервисов
builder.Services.AddControllersWithViews();

string mariadbCS = builder.Configuration.GetConnectionString("DefaultConnection");

builder.Services.AddDbContext<CalculatorContext>(options =>
{
    options.UseMySql(
        mariadbCS,
        new MySqlServerVersion(new Version(10, 5, 15))
    );
});

builder.Services.AddRazorPages();

var app = builder.Build();

// Настройка HTTP-конвейера
if (!app.Environment.IsDevelopment())
{
    app.UseExceptionHandler("/Calculator/Error");
    app.UseHsts();
}

app.UseHttpsRedirection();
app.UseStaticFiles();

app.UseRouting();

app.UseAuthorization();

app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Calculator}/{action=Index}/{id?}");

app.Run();
```